

BACKEND

Giải thích API endpoint là gì?

Một API endpoint là một URL cụ thể đóng vai trò như điểm truy cập vào một dịch vụ hoặc một chức năng nhất định bên trong dịch vụ đó. Thông qua API endpoint, các ứng dụng phía client có thể tương tác với server bằng cách gửi các request (đôi khi kèm theo dữ liệu dưới dạng payload) và nhận lại response từ server. Thông thường, mỗi endpoint sẽ được ánh xạ tương ứng với một tính năng riêng biệt ở phía server.

Bạn có thể giải thích sự khác nhau giữa cơ sở dữ liệu SQL và NoSQL không?

Cơ sở dữ liệu SQL (hay còn gọi là cơ sở dữ liệu quan hệ) dựa trên một schema (lược đồ hay cấu trúc) được định nghĩa sẵn cho dữ liệu. Mỗi khi mô tả một bản ghi hoặc một bảng trong cơ sở dữ liệu, bạn phải xác định rõ định dạng của nó, bao gồm tên bảng và các trường dữ liệu.

Ngược lại, trong các cơ sở dữ liệu NoSQL, không tồn tại schema cố định, vì vậy dữ liệu không bị ràng buộc bởi một cấu trúc được định nghĩa trước. Thông thường, dữ liệu được lưu trữ dưới dạng các collection gồm nhiều bản ghi, và các bản ghi này không bắt buộc phải có cùng cấu trúc, ngay cả khi về mặt khái niệm chúng đại diện cho cùng một loại thông tin.

RESTful API là gì, và các nguyên tắc cốt lõi của nó là gì?

Để một API được xem là RESTful (tức là tuân thủ các nguyên tắc của REST), nó cần đáp ứng một số yêu cầu cơ bản. Trước hết, API phải tuân theo kiến trúc client-server, nghĩa là phía client và phía server tách biệt rõ ràng (điều này vốn có ở hầu hết các dịch vụ dựa trên HTTP). API cũng phải cung cấp một giao diện thống nhất (uniform interface), trong đó mỗi tài nguyên cần được định danh rõ ràng thông qua URI (Unique Resource Identification), có thể được thay đổi thông qua biểu diễn của chính tài nguyên đó, và các thông điệp trao đổi phải tự mô tả, tức là chứa đủ thông tin để hiểu và xử lý chúng. Ngoài ra, client phải có khả năng khám phá các hành động có thể thực hiện với tài nguyên hiện tại dựa trên phản hồi từ server, cơ chế này được gọi là HATEOAS (Hypermedia as the Engine of Application State).

Bên cạnh đó, API RESTful phải stateless, tức là mỗi request gửi lên server đều phải chứa đầy đủ thông tin cần thiết để xử lý, không phụ thuộc vào trạng thái của các request trước đó. Hệ thống cũng nên được thiết kế theo mô hình layered system, nghĩa là client và server không nhất thiết phải kết nối trực tiếp với nhau mà có thể thông qua các lớp trung gian, nhưng điều này không được ảnh hưởng đến cách chúng giao tiếp. Các tài nguyên cần hỗ trợ cache, cho phép client hoặc server lưu trữ tạm thời dữ liệu để tăng hiệu năng. Cuối cùng, một tính năng tùy chọn của REST là Code on Demand, trong đó server có thể gửi mã về cho client để client thực thi khi cần.

Bạn có thể mô tả một chu trình request/response HTTP điển hình không?

Giao thức HTTP có cấu trúc rất chặt chẽ và bao gồm một tập các bước được định nghĩa rõ ràng. Trước tiên, client mở kết nối bằng cách thiết lập một kết nối TCP tới server;

cổng mặc định là 80 đối với HTTP và 443 đối với HTTPS (kết nối được bảo mật). Sau đó, client gửi request HTTP đến server, trong đó request bao gồm phương thức HTTP (như GET, POST, PUT, DELETE,...), một URI (Uniform Resource Identifier) để xác định vị trí tài nguyên trên server, phiên bản HTTP (thường là HTTP/1.1 hoặc HTTP/2), một tập headers chứa các thông tin bổ sung liên quan đến request, và body (không bắt buộc) dùng để gửi dữ liệu khi cần.

Khi request được gửi đi, server xử lý request và chuẩn bị phản hồi. Tiếp theo, server gửi HTTP response trở lại cho client thông qua kết nối đã mở. Response này bao gồm phiên bản HTTP, status code mô tả kết quả xử lý request, một tập headers chứa dữ liệu bổ sung, và body (tùy chọn) tương tự như trong request. Cuối cùng, kết nối được đóng lại; tuy nhiên, với các phiên bản HTTP mới hơn, có thể giữ kết nối mở để tiếp tục trao đổi nhiều request và response liên tiếp giữa client và server.

Bạn sẽ xử lý việc upload file trong một ứng dụng web như thế nào?

Từ góc nhìn của một lập trình viên backend, khi xử lý việc tải tệp (file upload), có một số yếu tố quan trọng cần được cân nhắc, bất kể bạn đang sử dụng ngôn ngữ lập trình nào. Trước hết, cần thực hiện kiểm tra và xác thực ở phía server, bao gồm việc đảm bảo kích thước tệp nằm trong giới hạn cho phép và tệp thuộc đúng loại yêu cầu; có thể tham khảo thêm các hướng dẫn của OWASP để hiểu rõ hơn về vấn đề bảo mật này. Bên cạnh đó, việc sử dụng kênh truyền an toàn là bắt buộc, tức là quá trình tải tệp phải được thực hiện thông qua kết nối HTTPS.

Ngoài ra, cần tránh trùng tên tệp bằng cách đổi tên tệp sau khi tải lên sao cho tên mới là duy nhất trong hệ thống, vì nếu không sẽ dễ gây ra lỗi ứng dụng khi lưu trữ tệp. Cuối cùng, bạn nên lưu trữ metadata của các tệp (ví dụ: tên gốc, kích thước, loại tệp, thời gian tải lên...) trong cơ sở dữ liệu hoặc một nơi phù hợp khác để tiện cho việc quản lý và cung cấp thêm thông tin cho người dùng. Đặc biệt, nếu bạn đổi tên tệp vì lý do bảo mật hoặc để tránh trùng lặp, hãy đảm bảo vẫn lưu lại tên tệp gốc để có thể sử dụng khi người dùng cần tải tệp về sau.

Bạn sẽ viết những loại kiểm thử (test) nào cho một API endpoint mới?

Là các lập trình viên backend, chúng ta cần suy nghĩ và nắm rõ những loại kiểm thử khác nhau có thể áp dụng trong quá trình phát triển hệ thống. Trước hết là unit test, trong đó bạn chỉ nên kiểm thử phần logic liên quan thông qua giao diện công khai của mã nguồn, tránh việc kiểm thử các phương thức private hoặc những hàm không thể truy cập bên trong module. Unit test nên tập trung vào logic nghiệp vụ cốt lõi và không nên kiểm thử các đoạn mã phụ thuộc vào dịch vụ bên ngoài như cơ sở dữ liệu, ổ đĩa hay các hệ thống khác ngoài phạm vi đoạn code đang được kiểm thử.

Tiếp theo là integration test, loại kiểm thử này nhằm kiểm tra toàn bộ endpoint thông qua giao diện công khai của nó, thường là endpoint HTTP thực tế, để xem hệ thống tích hợp và hoạt động như thế nào với các dịch vụ bên ngoài mà nó sử dụng, chẳng hạn như cơ sở dữ liệu hoặc một API khác. Cuối cùng là load testing hoặc performance testing, nơi bạn đánh giá cách endpoint mới hoạt động khi phải chịu tải lớn hoặc áp lực cao.

Việc này có thể không bắt buộc trong mọi trường hợp, tùy vào loại API bạn xây dựng, nhưng nhìn chung đây là một bước nên thực hiện ở giai đoạn cuối của quá trình phát triển, trước khi triển khai endpoint mới lên môi trường production.

Mô tả cách quản lý phiên (session management) hoạt động trong các ứng dụng web

Quy trình quản lý session trong các ứng dụng web ở mức tổng quan thường diễn ra như sau: Session được tạo ra ngay từ lần tương tác đầu tiên của người dùng với hệ thống, thường là khi đăng nhập, lúc này backend sẽ sinh ra một session ID duy nhất, lưu trữ và trả lại cho người dùng để sử dụng cho các request sau này. Tiếp theo, thông tin session cần được lưu trữ ở một nơi nào đó, có thể là trong bộ nhớ hoặc trong cơ sở dữ liệu, và được đánh index theo session ID đã tạo; giải pháp tối ưu thường là dùng cơ sở dữ liệu có hiệu năng I/O cao như Redis để các service có thể scale độc lập với dữ liệu session. Sau đó, session ID được gửi về phía client, phổ biến nhất là thông qua cookie, backend sẽ thiết lập cookie chứa session ID và frontend có thể đọc một cách an toàn để sử dụng khi cần. Ở các request tiếp theo, client sẽ gửi session ID này kèm theo để định danh với backend. Backend sẽ dựa vào session ID nhận được để truy xuất dữ liệu session đã lưu trữ. Cuối cùng, khi hết hạn hoặc do một hành động nào đó của người dùng, session ID sẽ bị xóa, kéo theo dữ liệu session cũng bị mất hoặc bị loại bỏ khỏi cơ sở dữ liệu, và điều này đánh dấu việc kết thúc tương tác giữa client và server trong session đó.

Bạn tiếp cận việc quản lý phiên bản API (API versioning) trong các dự án của mình như thế nào?

Có nhiều cách khác nhau để xử lý việc versioning cho API, nhưng phổ biến nhất là hai phương pháp sau. Cách thứ nhất là đưa phiên bản vào trong URL, có thể dưới dạng tham số URL (ví dụ: /your-api/users?v=1) hoặc như một phần của đường dẫn (ví dụ: /v1/your-api/users); với cách này, phiên bản API được hiển thị rõ ràng để lập trình viên sử dụng API có thể dễ dàng nhận biết. Cách thứ hai là sử dụng custom header, trong đó client gửi kèm một header riêng (chẳng hạn như api-version) để chỉ rõ phiên bản API mà họ muốn sử dụng.

Bạn bảo vệ máy chủ khỏi các cuộc tấn công SQL Injection như thế nào?

Có nhiều cách để bảo vệ cơ sở dữ liệu quan hệ khỏi các cuộc tấn công SQL Injection, nhưng phổ biến nhất có thể kể đến ba phương pháp sau. Thứ nhất là sử dụng prepared statements với các truy vấn có tham số hóa; đây được xem là cách hiệu quả nhất vì nó do thư viện hoặc framework xử lý, lập trình viên chỉ cần viết câu truy vấn với các placeholder cho dữ liệu, sau đó cung cấp dữ liệu thực tế ở một bước riêng biệt. Thứ hai là sử dụng ORM (Object-Relational Mapping), các framework này giúp trừu tượng hóa việc tương tác với cơ sở dữ liệu và tự động sinh câu lệnh SQL, đồng thời xử lý sẵn các vấn đề bảo mật liên quan. Cuối cùng là escape dữ liệu thủ công, trong đó bạn cần xử lý và loại bỏ hoặc mã hóa các ký tự đặc biệt có thể làm hỏng câu truy vấn SQL; việc duy trì một danh sách các ký tự bị cấm để escape và xử lý chúng một cách tự động là một cách tiếp cận hợp lý.

Giải thích khái niệm “stateless” (không trạng thái) trong HTTP và cách nó ảnh hưởng đến các dịch vụ backend

HTTP về bản chất là một giao thức stateless (không lưu trạng thái), nghĩa là mỗi request đều độc lập và không liên quan đến bất kỳ request nào trước đó, kể cả khi chúng đến từ cùng một client. Điều này ảnh hưởng trực tiếp đến các dịch vụ web phía backend, bởi nếu ứng dụng cần duy trì trạng thái (state) giữa các lần tương tác thì backend buộc phải tự triển khai các cơ chế quản lý trạng thái riêng, chẳng hạn như sử dụng session, cookie hoặc token.

Containerization là gì, và nó mang lại lợi ích gì cho việc phát triển backend?

Đây là một phương pháp ảo hóa nhẹ dùng để đóng gói ứng dụng cùng với toàn bộ các dependency của nó, nhằm đảm bảo môi trường chạy luôn nhất quán trên các hệ thống khác nhau. Điều này đặc biệt có lợi cho phát triển backend vì nó mang lại tính cô lập và khả năng di chuyển cao, giúp đơn giản hóa quá trình triển khai và giảm thiểu xung đột giữa các phiên bản phần mềm cũng như cấu hình hệ thống.

Bạn sẽ thực hiện những biện pháp nào để bảo mật một API mới được phát triển?

Có nhiều cách để bảo mật một API, trong đó phổ biến nhất là bổ sung cơ chế xác thực như OAuth, JWT, Bearer token hoặc xác thực dựa trên session; sử dụng HTTPS để mã hóa dữ liệu truyền tải giữa client và server; cấu hình chính sách CORS chặt chẽ nhằm ngăn chặn các yêu cầu không mong muốn; và xây dựng logic phân quyền mạnh mẽ để đảm bảo client chỉ có thể truy cập những tài nguyên mà họ được phép sử dụng.

Bạn sẽ mở rộng (scale) một ứng dụng backend như thế nào khi lưu lượng truy cập tăng đột biến?

Cách phổ biến nhất để mở rộng một ứng dụng backend khi lưu lượng truy cập tăng đột biến là triển khai nhiều instance của ứng dụng phía sau một bộ cân bằng tải (load balancer), và khi xảy ra sự gia tăng lưu lượng, chỉ cần bổ sung thêm các instance mới. Cách tiếp cận này được gọi là mở rộng theo chiều ngang (horizontal scaling) và hoạt động hiệu quả nhất khi ứng dụng backend là không trạng thái (stateless).

Bạn sử dụng những công cụ và kỹ thuật nào để gỡ lỗi (debug) một ứng dụng backend?

Nếu ứng dụng backend đang được debug chạy trên máy phát triển cục bộ (local dev machine), một giải pháp đơn giản là sử dụng trực tiếp IDE. Hầu hết các IDE hiện đại như IntelliJ, Eclipse và nhiều công cụ khác đều đã tích hợp sẵn các tính năng gỡ lỗi (debug). Tuy nhiên, nếu ứng dụng backend đang chạy trên server, bạn sẽ cần áp dụng các kỹ thuật khác, chẳng hạn như ghi log bằng các thư viện logging. Ngoài ra, bạn cũng có thể sử dụng những công cụ nâng cao hơn như JProfiler hoặc New Relic để theo dõi, phân tích và xử lý các vấn đề phát sinh.

Bạn đảm bảo mã backend của mình dễ bảo trì và dễ hiểu như thế nào?

Mẹo ở đây là tuân thủ các best practices và chuẩn lập trình, chẳng hạn như thiết kế mã theo hướng mô-đun để dễ bảo trì và mở rộng, tuân theo các quy ước đặt tên thống nhất nhằm tăng tính dễ đọc, thêm chú thích trong mã nguồn để giải thích mục đích và cách hoạt động của các đoạn code quan trọng, thực hiện refactor thường xuyên để kiểm soát và giảm thiểu technical debt, đảm bảo các thông báo xử lý lỗi được sử dụng nhất quán trên toàn bộ nền tảng, và thực hiện unit test cho tất cả các đoạn mã đã viết để đảm bảo chất lượng và độ ổn định của hệ thống.

Mô tả cách bạn sẽ triển khai tìm kiếm toàn văn (full-text search) trong một cơ sở dữ liệu.

Bạn có thể sử dụng sẵn chức năng tìm kiếm toàn văn (full-text search) của cơ sở dữ liệu như MySQL, PostgreSQL hoặc thậm chí là Elasticsearch. Tuy nhiên, nếu muốn tự triển khai cơ chế này, trước hết bạn cần tiền xử lý dữ liệu văn bản cần tìm kiếm và chuẩn hóa nó bằng cách áp dụng các kỹ thuật như tách từ (tokenization), rút gọn từ (stemming) và loại bỏ các từ dừng (stop words); sau đó xây dựng một inverted index để liên kết mỗi từ duy nhất với các bản ghi có chứa từ đó; tiếp theo là tạo giao diện tìm kiếm và chuẩn hóa dữ liệu đầu vào của người dùng theo cùng cách đã áp dụng cho dữ liệu văn bản ban đầu; rồi thực hiện tìm kiếm từng từ trong cơ sở dữ liệu; cuối cùng, sắp xếp kết quả bằng cách xây dựng một cơ chế chấm điểm dựa trên nhiều yếu tố khác nhau, chẳng hạn như tần suất xuất hiện của từ.

Bạn sẽ tiếp cận việc xử lý theo lô (batch processing) như thế nào trong một ứng dụng backend xử lý nhiều dữ liệu?

Lựa chọn tốt nhất trong trường hợp này là sử dụng một framework xử lý theo lô (batch processing) như Hadoop hoặc Spark. Các công cụ này đã được thiết kế sẵn để xử lý khối lượng dữ liệu khổng lồ một cách song song, giúp tăng hiệu năng và khả năng mở rộng khi làm việc với dữ liệu lớn.

Bạn có thể giải thích việc sử dụng và lợi ích của hàng đợi thông điệp (message queue) trong một hệ thống phân tán không?

Trong một hệ thống phân tán, hàng đợi thông điệp (message queue) có thể đóng vai trò là thành phần cốt lõi của kiến trúc phản ứng (reactive architecture). Mỗi dịch vụ có thể phát sinh và lắng nghe các sự kiện đến từ hàng đợi, nhờ đó khi sự kiện xuất hiện, các dịch vụ có thể phản ứng ngay lập tức mà không cần phải chủ động liên tục truy vấn (poll) các dịch vụ khác để chờ phản hồi.

Bạn sẽ sử dụng những chiến lược nào để quản lý kết nối cơ sở dữ liệu trong tình huống tải cao (high-load)?

Trong các tình huống hệ thống phải chịu tải cao, có một số cách mà lập trình viên có thể áp dụng để cải thiện hiệu năng của kết nối cơ sở dữ liệu. Trước hết, việc sử dụng connection pool để tái sử dụng các kết nối sẽ giúp giảm thời gian cần thiết để thiết lập

một kết nối mới. Bên cạnh đó, cân bằng tải lưu lượng truy vấn cơ sở dữ liệu giữa nhiều máy chủ database khác nhau cũng giúp phân tán tải và tránh quá tải tại một điểm. Cuối cùng, việc tối ưu hóa các câu truy vấn sẽ làm giảm thời gian mỗi kết nối được sử dụng, từ đó giúp khai thác tài nguyên hiệu quả hơn và rút ngắn thời gian duy trì các kết nối đang hoạt động.

Bạn sẽ thiết lập một pipeline tích hợp liên tục/triển khai liên tục (CI/CD) cho các dịch vụ backend như thế nào?

Khi thiết lập các pipeline Continuous Integration (CI) và Continuous Delivery (CD), có nhiều yếu tố quan trọng cần được cân nhắc. Trước hết, nên sử dụng hệ thống quản lý mã nguồn làm điểm kích hoạt cho toàn bộ quy trình (ví dụ như Git), trong đó các pipeline build cho dịch vụ backend sẽ được chạy mỗi khi bạn đẩy mã nguồn lên một nhánh cụ thể. Tiếp theo, cần lựa chọn nền tảng CI/CD phù hợp với nhu cầu của dự án, chẳng hạn như GitHub Actions, GitLab CI/CD, CircleCI hoặc các công cụ tương tự. Ngoài ra, hãy đảm bảo rằng bạn có các bài kiểm thử đơn vị (unit test) tự động có thể chạy bên trong các pipeline này. Việc triển khai tự động chỉ nên diễn ra khi tất cả các bài kiểm thử đều chạy thành công; nếu không, pipeline phải thất bại để ngăn chặn mã lỗi được đưa vào các môi trường khác. Bên cạnh đó, nên sử dụng kho lưu trữ artifact như JFrog Artifactory hoặc Nexus Repository để lưu trữ các dịch vụ đã được build thành công. Cuối cùng, cần cân nhắc xây dựng một chiến lược rollback để có thể quay lại phiên bản trước trong trường hợp có sự cố xảy ra và phiên bản dịch vụ vừa triển khai gặp lỗi.

Bạn có thể mô tả một chiến lược cache phân tán (distributed caching) cho một ứng dụng có tính sẵn sàng cao (high availability) không?

Trong kịch bản này, bạn cần xem xét một số điểm quan trọng. Trước hết, hãy triển khai một cụm máy chủ, trong đó tất cả các máy chủ đều hoạt động như một hệ thống cache phân tán. Cần áp dụng cơ chế sharding dữ liệu để phân phối dữ liệu đồng đều giữa các máy chủ cache, đồng thời sử dụng thuật toán consistent hashing nhằm giảm thiểu việc tái tổ chức cache khi có máy chủ tham gia hoặc rời khỏi cụm. Bên cạnh đó, nên bổ sung cơ chế sao chép cache (cache replication) để tạo tính dư thừa dữ liệu trong trường hợp xảy ra sự cố, qua đó giúp hệ thống cache phân tán có khả năng chịu lỗi tốt hơn. Cuối cùng, việc vô hiệu hóa và làm mới cache (cache invalidation) là bắt buộc đối với bất kỳ giải pháp cache nào, bởi nếu dữ liệu không được cập nhật thường xuyên, nó sẽ nhanh chóng trở nên lỗi thời.

Bạn có thể sử dụng những phương pháp nào để quản lý các tác vụ nền (background tasks) trong ứng dụng của mình?

Việc xử lý các tác vụ chạy nền phụ thuộc rất nhiều vào tech stack mà bạn đang sử dụng cũng như bản chất của những tác vụ đó. Vì vậy, có khá nhiều lựa chọn khác nhau. Một cách phổ biến là sử dụng các hàng đợi tác vụ (task queue) như RabbitMQ hoặc Amazon SQS, cho phép bạn chạy các worker ở chế độ nền như những tiến trình phụ trong khi ứng dụng chính vẫn tiếp tục hoạt động bình thường. Ngoài ra, bạn có thể dùng các

framework chuyên cho background job, chẳng hạn như Celery đối với Python hoặc Sidekiq đối với Ruby. Trong những trường hợp đơn giản hơn, bạn cũng có thể chỉ cần dựa vào các cron job để thực thi tác vụ theo lịch. Cuối cùng, nếu ngôn ngữ lập trình bạn dùng hỗ trợ, bạn có thể sử dụng thread hoặc worker để chạy các tác vụ nền ngay trong cùng một ứng dụng.

Bạn xử lý việc mã hóa và giải mã dữ liệu như thế nào trong một ứng dụng chú trọng đến quyền riêng tư?

Đối với loại ứng dụng này, bạn cần phân biệt rõ giữa “data at rest” (dữ liệu tĩnh) và “data in transit” (dữ liệu đang truyền). “Data at rest” dùng để chỉ dữ liệu khi nó đang được lưu trữ trong cơ sở dữ liệu hoặc bất kỳ hệ thống lưu trữ nào khác, còn “data in transit” mô tả dữ liệu trong quá trình được truyền giữa các dịch vụ backend với nhau hoặc giữa server và client.

Đối với “data in transit”, bạn cần đảm bảo rằng việc kết nối diễn ra thông qua các kênh an toàn và được mã hóa, chẳng hạn như HTTPS. Còn với “data at rest”, hãy sử dụng các thuật toán mã hóa mạnh như AES, RSA hoặc ECC, đồng thời đảm bảo các khóa mã hóa liên quan được lưu trữ ở nơi an toàn, ví dụ như trong các công cụ quản lý bí mật (secrets management) chuyên dụng hoặc các dịch vụ quản lý khóa (KMS).

Webhook là gì và bạn đã triển khai chúng như thế nào trong các dự án trước đây?

Webhook là các callback HTTP do người dùng tự định nghĩa, được kích hoạt bởi một sự kiện cụ thể xảy ra bên trong hệ thống. Chúng chủ yếu được sử dụng để thông báo kết quả của các tác vụ bất đồng bộ nhiều bước, nhằm tránh việc phải giữ một kết nối HTTP mở liên tục. Khi triển khai webhook, cần lưu ý một số điểm quan trọng như: xác định rõ sự kiện nào sẽ kích hoạt webhook và loại payload đi kèm; tạo endpoint HTTP phù hợp để xử lý request mong đợi, đặc biệt là phần dữ liệu payload (ví dụ, nếu webhook nhận dữ liệu thì nên dùng phương thức POST, còn nếu không thì có thể dùng GET); và cuối cùng là đảm bảo yếu tố bảo mật bằng cách áp dụng các biện pháp an ninh cần thiết để endpoint webhook không bị lợi dụng.

Những yếu tố nào cần được xem xét để tuân thủ GDPR trong một hệ thống backend?

Những điểm quan trọng cần được xem xét bao gồm: chỉ thu thập những dữ liệu thực sự cần thiết và đúng với những gì bạn đã thông báo cho người dùng, bởi để tuân thủ GDPR, bạn phải xin sự đồng ý của người dùng và nêu rõ chính xác các loại dữ liệu sẽ được thu thập, vì vậy hãy tập trung vào các dữ liệu đó và không thu thập thêm ngoài phạm vi đã cam kết; đảm bảo an toàn cho dữ liệu, vì theo quy định, dữ liệu phải được bảo mật cả khi đang truyền tải lẫn khi được lưu trữ, đồng thời cần thực hiện các đợt kiểm tra bảo mật định kỳ để duy trì mức độ an toàn cao; và cuối cùng, người dùng có quyền đối với dữ liệu của họ, do đó bạn cần cung cấp các endpoint hoặc dịch vụ phù hợp để họ có thể đọc, chỉnh sửa hoặc thậm chí yêu cầu xóa dữ liệu khi cần.

Giải thích cách bạn sẽ xử lý các tiến trình chạy lâu (long-running processes) trong các yêu cầu web.

Đối với các yêu cầu web kích hoạt những tiến trình chạy trong thời gian dài, giải pháp tốt nhất là triển khai kiến trúc phản ứng (reactive architecture). Điều này có nghĩa là khi một dịch vụ nhận được yêu cầu, nó sẽ chuyển yêu cầu đó thành một thông điệp trong hàng đợi tin nhắn (message queue), và tiến trình chạy lâu sẽ lấy thông điệp này để xử lý khi sẵn sàng. Trong thời gian đó, phía client gửi yêu cầu sẽ nhận được phản hồi ngay lập tức để xác nhận rằng yêu cầu đang được xử lý. Bản thân client cũng có thể được kết nối với hàng đợi tin nhắn (hoặc thông qua một proxy) và chờ sự kiện “ready” kèm theo payload của nó khi quá trình xử lý hoàn tất.

Thảo luận về việc triển khai giới hạn tần suất (rate limiting) để bảo vệ API khỏi bị lạm dụng.

Để triển khai cơ chế giới hạn tần suất truy cập (rate limiting), bạn cần lưu ý một số điểm quan trọng. Trước hết, hãy xác định rõ giới hạn, tức là quy định chính xác số lượng yêu cầu mà một client được phép gửi, có thể tính theo số request mỗi giây, mỗi phút hoặc mỗi ngày. Tiếp theo, bạn cần lựa chọn chiến lược giới hạn phù hợp bằng cách áp dụng một thuật toán rate limiting như fixed window counter, sliding log window, token bucket hoặc leaky bucket. Sau đó, các bộ đếm hoặc thông tin liên quan đến số lượng yêu cầu cần được lưu trữ ở một nơi có tốc độ truy cập cao, chẳng hạn như Redis, để theo dõi số request hoặc mốc thời gian của từng client. Khi client vượt quá giới hạn cho phép, hệ thống nên phản hồi bằng một mã trạng thái tiêu chuẩn, ví dụ như 429, cho biết có “Too Many Requests”. Nếu muốn nâng cao hơn, bạn có thể cân nhắc sử dụng một API Gateway đã tích hợp sẵn chức năng này hoặc bổ sung cơ chế xử lý các đợt tăng lưu lượng đột ngột nhằm tránh phạt những client chỉ vượt giới hạn một cách nhỏ và không thường xuyên.

Bạn đo lường (instrument) và giám sát hiệu năng của các ứng dụng backend như thế nào?

Một cách rất hiệu quả để theo dõi hiệu năng của các ứng dụng backend là sử dụng các hệ thống quản lý hiệu năng ứng dụng (Application Performance Management – APM) như New Relic, AppDynamics hoặc Dynatrace. Những công cụ này sẽ theo dõi hoạt động và hiệu suất của ứng dụng, đồng thời cung cấp cho bạn cái nhìn chi tiết về các điểm nghẽn tiềm ẩn trong hệ thống với rất ít công sức triển khai và vận hành từ phía bạn.

Microservices là gì, và bạn sẽ phân tách một hệ thống nguyên khối (monolith) thành các microservices như thế nào?

Microservices là một phong cách kiến trúc phần mềm cho phép bạn tổ chức các ứng dụng backend dưới dạng một tập hợp các dịch vụ độc lập, trong đó mỗi dịch vụ đảm nhiệm một nhu cầu nghiệp vụ cụ thể. Khi muốn phân rã một hệ thống nguyên khối (monolith) thành các microservices, bạn cần lưu ý một số điểm quan trọng: trước hết,

hãy xác định các ranh giới logic trong hệ thống monolith, vì bên trong nó thường xử lý nhiều trách nhiệm và nhiều loại tài nguyên khác nhau, từ đó tìm ra ranh giới để biết dịch vụ này kết thúc và dịch vụ khác bắt đầu ở đâu; tiếp theo, định nghĩa các dịch vụ dựa trên những ranh giới đã xác định và đồng thời tách biệt nhu cầu dữ liệu, có thể là thành nhiều bảng khác nhau hoặc thậm chí là các cơ sở dữ liệu riêng khi phù hợp; sau đó, tiến hành tái cấu trúc monolith một cách dần dần, trích xuất phần logic cần thiết cho từng microservice thành các dự án độc lập; cuối cùng, khi quá trình hoàn tất, hệ thống monolith ban đầu sẽ không còn cần thiết nữa và mỗi microservice sẽ có pipeline triển khai cũng như kho mã nguồn riêng biệt, độc lập với nhau.

Bạn đã quản lý các phụ thuộc API (API dependencies) trong các hệ thống backend như thế nào?

Một cách rất hiệu quả để xử lý các phụ thuộc API trong hệ thống backend là tận dụng cơ chế versioning (quản lý phiên bản API). Với phương pháp đơn giản này, bạn có thể đảm bảo rằng các hệ thống của mình luôn sử dụng đúng phiên bản API cần thiết, ngay cả khi API đó tồn tại nhiều phiên bản khác nhau. Đồng thời, cách làm này còn cho phép nhiều hệ thống backend sử dụng các phiên bản khác nhau của cùng một API mà không gây ra rủi ro về sự không nhất quán hay việc các bản cập nhật làm hỏng hoặc ảnh hưởng đến hoạt động của hệ thống.

Mô tả khái niệm tính nhất quán cuối cùng (eventual consistency) và những tác động của nó trong các hệ thống backend.

Eventual consistency là một mô hình nhất quán được sử dụng trong các hệ thống phân tán. Mô hình này đảm bảo rằng bất kỳ dữ liệu nào được ghi vào hệ thống phân tán cuối cùng cũng sẽ trở nên nhất quán, tức là tất cả các máy chủ sẽ có cùng một phiên bản dữ liệu, thay vì yêu cầu tính nhất quán ngay lập tức như một số mô hình khác. Đối với các hệ thống backend, điều này đồng nghĩa với việc cần có cơ chế đồng bộ dữ liệu giữa tất cả các thành phần của hệ thống phân tán, đồng thời có thể phải xử lý và giải quyết xung đột dữ liệu khi các phần khác nhau của hệ thống đang làm việc với những phiên bản khác nhau của cùng một bản ghi dữ liệu.

Reverse proxy là gì, và nó hữu ích như thế nào trong phát triển backend?

Reverse proxy là một máy chủ đứng phía trước nhiều máy chủ khác và có nhiệm vụ chuyển hướng lưu lượng truy cập đến các máy chủ web đó dựa trên những quy tắc logic khác nhau. Ví dụ, bạn có thể có hai máy chủ web, một phục vụ khách hàng của doanh nghiệp và một phục vụ nhân viên nội bộ. Khi đó, reverse proxy có thể được cấu hình để chuyển hướng yêu cầu đến máy chủ phù hợp dựa trên giá trị của một header trong request hoặc dựa trên URL mà client đang truy cập. Reverse proxy rất hữu ích trong phát triển backend vì nó cho phép thực hiện nhiều chức năng quan trọng, chẳng hạn như cân bằng tải giữa nhiều instance của cùng một dịch vụ backend, cung cấp thêm một lớp bảo mật bằng cách che giấu vị trí thực của các dịch vụ backend và xử lý các cuộc tấn công như DDoS, hỗ trợ cache nội dung để giảm tải cho các máy chủ web, đồng thời cho

phép thay đổi hoặc chuyển đổi các dịch vụ backend mà không làm ảnh hưởng đến các URL công khai mà người dùng đang sử dụng.

Bạn sẽ xử lý trạng thái phiên (session state) như thế nào trong một môi trường ứng dụng có cân bằng tải (load-balanced)?

Trong một kịch bản ứng dụng có cân bằng tải, vấn đề lớn nhất liên quan đến trạng thái phiên (session state) là nếu hệ thống backend lưu dữ liệu phiên trong bộ nhớ (in-memory), thì các request tiếp theo từ cùng một client phải luôn được chuyển đến đúng server ban đầu; nếu không, dữ liệu phiên sẽ bị phân mảnh và trở nên vô dụng. Có hai cách chính để giải quyết vấn đề này. Cách thứ nhất là sticky session, cho phép cấu hình load balancer để luôn chuyển các request từ cùng một client đến cùng một server; nhược điểm của phương pháp này là lưu lượng truy cập có thể không được phân phối đồng đều giữa các instance backend. Cách thứ hai là centralized session store, tức là đưa dữ liệu session ra khỏi các backend service và lưu vào một kho dữ liệu tập trung mà tất cả các instance đều có thể truy cập; cách này giúp load balancer hoạt động đơn giản hơn nhưng lại yêu cầu thêm logic xử lý và làm hệ thống phức tạp hơn. Việc lựa chọn chiến lược nào phụ thuộc vào yêu cầu kỹ thuật và bối cảnh cụ thể của hệ thống bạn đang xây dựng.

Sao chép cơ sở dữ liệu (database replication) là gì, và nó có thể được sử dụng để tăng khả năng chịu lỗi (fault tolerance) như thế nào?

Sao chép cơ sở dữ liệu (database replication) là việc nhân bản dữ liệu giữa nhiều instance của cùng một cơ sở dữ liệu. Trong mô hình này, thường sẽ có một cơ sở dữ liệu đóng vai trò master, nơi tất cả các client kết nối vào để ghi dữ liệu, còn các cơ sở dữ liệu còn lại đóng vai trò slave, chỉ nhận các bản cập nhật khi dữ liệu được thêm mới hoặc thay đổi. Cách tiếp cận này mang lại hai ý nghĩa quan trọng về khả năng chịu lỗi: thứ nhất, cụm cơ sở dữ liệu có thể tiếp tục hoạt động khi máy chủ master gặp sự cố bằng cách nâng cấp một slave lên làm master mà không làm mất dữ liệu; thứ hai, các slave có thể được sử dụng như các máy chủ chỉ đọc, giúp tăng số lượng yêu cầu đọc có thể xử lý mà không ảnh hưởng đến hiệu năng của cơ sở dữ liệu chính.

Mô tả việc sử dụng chiến lược triển khai blue-green trong các dịch vụ backend.

Chiến lược blue-green là cách triển khai trong đó tồn tại hai môi trường production giống hệt nhau. Một môi trường đang trực tiếp phục vụ lưu lượng người dùng thực tế, trong khi môi trường còn lại được chuẩn bị để cập nhật phiên bản mới hoặc đơn giản là ở trạng thái chờ, đóng vai trò dự phòng. Khi cần phát hành bản mới, lưu lượng có thể được chuyển sang môi trường đã cập nhật một cách nhanh chóng, giúp giảm thiểu thời gian gián đoạn và rủi ro.

Bạn có thể giải thích các mô hình nhất quán trong cơ sở dữ liệu phân tán (ví dụ: định lý CAP) không?

Định lý CAP cho rằng trong các hệ cơ sở dữ liệu phân tán, không thể đồng thời đảm bảo cả ba yếu tố sau, mà chỉ có thể chọn tối đa hai trong số chúng. Thứ nhất là tính nhất

quán dữ liệu (Consistency), nghĩa là mọi thao tác đọc luôn trả về kết quả mới nhất của thao tác ghi, điều này đặc biệt quan trọng vì dữ liệu được nhân bản trên nhiều máy chủ và cần được đồng bộ gần như ngay lập tức. Thứ hai là tính sẵn sàng (Availability), tức là mọi yêu cầu gửi đến hệ thống đều luôn nhận được một phản hồi hợp lệ. Thứ ba là khả năng chịu phân vùng (Partition tolerance), nghĩa là hệ thống vẫn tiếp tục hoạt động và không mất dữ liệu ngay cả khi xảy ra sự cố gián đoạn mạng cục bộ. Ví dụ, nếu một hệ thống đảm bảo được tính nhất quán và tính sẵn sàng cao, thì nó sẽ không thể chịu được các sự cố phân vùng mạng. Ngược lại, nếu hệ thống ưu tiên tính sẵn sàng và khả năng chịu phân vùng, thì nó sẽ không thể đảm bảo tính nhất quán dữ liệu ngay lập tức.

Bạn quản lý việc migration schema (lược đồ cơ sở dữ liệu) như thế nào trong một môi trường triển khai liên tục (continuous delivery)?

Hai khía cạnh chính cần cân nhắc khi quản lý việc migration schema, đặc biệt trong các môi trường Continuous Deployment (CD), là theo dõi các thay đổi schema trong hệ thống quản lý mã nguồn (version control) và sử dụng các công cụ tự động hóa. Cụ thể, các file migration nên được lưu trữ và quản lý phiên bản cùng với mã nguồn ứng dụng, vì giữa phiên bản code và phiên bản schema của cơ sở dữ liệu có mối quan hệ trực tiếp. Bên cạnh đó, việc sử dụng các công cụ migration tự động như Flyway hoặc Liquibase sẽ giúp đơn giản hóa quy trình, đảm bảo tính nhất quán và tiêu chuẩn hóa quá trình cập nhật schema trên các môi trường khác nhau.

Những chiến lược nào tồn tại để xử lý tính bất biến (idempotency) trong thiết kế REST API?

Đối với các REST API, bạn có thể tận dụng các HTTP verb và định nghĩa những thao tác idempotent bằng cách sử dụng các verb vốn đã mang tính idempotent, chẳng hạn như GET, PUT và DELETE. Ngoài ra, bạn cũng có thể tự triển khai một cơ chế dựa trên khóa (key-based logic) để tránh việc lặp lại cùng một thao tác nhiều lần, trong trường hợp khóa do phía client gửi lên luôn giống nhau cho cùng một yêu cầu.

Mô tả việc triển khai một giải pháp đăng nhập một lần (Single Sign-On – SSO).

Ở mức độ tổng quan, các bước cần thiết để triển khai một giải pháp SSO (Single Sign-On) bao gồm việc lựa chọn một nhà cung cấp danh tính (Identity Provider – IdP) như Okta hoặc Keycloak. Sau đó, mỗi ứng dụng sẽ tích hợp với IdP này thông qua một giao thức SSO tiêu chuẩn như SAML, OpenID hoặc các giao thức tương đương khác. Trong lần truy cập đầu tiên của người dùng, ứng dụng sẽ kết nối với IdP để xác thực danh tính và nhận về một access token. Ở các yêu cầu tiếp theo, ứng dụng sẽ sử dụng token này và xác minh tính hợp lệ của nó thông qua IdP để đảm bảo người dùng đã được xác thực.

Giải thích cách bạn sẽ phát triển một hệ thống backend để xử lý các luồng dữ liệu từ thiết bị IoT.

Một kiến trúc thu thập và xử lý dữ liệu thời gian thực sẽ cần các thành phần sau: trước hết là sử dụng một dịch vụ ingestion dữ liệu có khả năng mở rộng như Kafka hoặc AWS

Kinesis, tương thích với các giao thức IoT tiêu chuẩn phổ biến như MQTT hoặc CoAP. Tiếp theo, dữ liệu được xử lý thông qua các công cụ xử lý thời gian thực như Apache Flink hoặc Spark Streaming để phân tích và biến đổi ngay khi dữ liệu được tạo ra. Cuối cùng, dữ liệu đã xử lý sẽ được lưu trữ trong một data lake có khả năng mở rộng, lý tưởng nhất là các hệ thống hỗ trợ dữ liệu chuỗi thời gian như InfluxDB, nhằm phục vụ cho việc truy vấn và phân tích lâu dài.

Bạn sẽ thiết kế kiến trúc backend như thế nào để hỗ trợ đồng bộ dữ liệu theo thời gian thực giữa các thiết bị?

Để hỗ trợ đồng bộ dữ liệu theo thời gian thực, bạn cần tìm cách tạo ra các kênh giao tiếp ổn định và hiệu quả giữa các thiết bị, đồng thời có giải pháp xử lý xung đột đồng bộ dữ liệu khi nhiều thiết bị cùng cố gắng thay đổi một bản ghi.

Về kênh giao tiếp, có thể sử dụng các kênh hai chiều dựa trên socket cho phép trao đổi dữ liệu theo thời gian thực, hoặc áp dụng mô hình pub/sub để phân phối dữ liệu hiệu quả giữa nhiều thiết bị, với các công cụ như Redis hoặc Kafka.

Đối với việc giải quyết xung đột dữ liệu, có thể sử dụng các thuật toán như Operational Transformation (OT) hoặc Conflict-Free Replicated Data Types (CRDTs) nhằm đảm bảo dữ liệu cuối cùng được đồng bộ một cách nhất quán trên tất cả các thiết bị.

Thảo luận về lợi ích và hạn chế của kiến trúc microservice trong các hệ thống backend.

Lợi ích của kiến trúc microservices bao gồm khả năng mở rộng cao, vì mỗi microservice có thể được scale độc lập với các service khác; tính linh hoạt về công nghệ, cho phép sử dụng các công nghệ khác nhau tùy theo nhu cầu cụ thể của từng microservice; và triển khai nhanh hơn, do mỗi microservice có thể được deploy riêng lẻ, giúp tăng tốc độ đưa các thay đổi lên môi trường production.

Tuy nhiên, kiến trúc này cũng có những hạn chế nhất định. Nó có thể dẫn đến độ phức tạp cao, khi hệ thống dựa trên microservices trở nên khó quản lý và điều phối trong một số trường hợp. Việc debug cũng trở nên khó khăn hơn, vì dữ liệu phải đi qua nhiều service khác nhau trong một request duy nhất. Ngoài ra, chi phí giao tiếp giữa các microservice có thể cao và phức tạp hơn so với cách tiếp cận monolithic truyền thống.

Bạn sẽ tiếp cận việc kiểm thử tải (load testing) cho một API backend như thế nào?

Trước hết, bạn cần hiểu rõ mục tiêu và thiết lập một môi trường kiểm thử phù hợp; lý tưởng nhất là môi trường này phải gần giống với môi trường production. Sau đó, hãy thiết kế và triển khai các bài kiểm thử bằng những công cụ đã chọn, chẳng hạn như JMeter, LoadRunner hoặc các công cụ tương tự. Tiếp theo, bạn tăng dần tải cho các bài kiểm thử, đồng thời theo dõi hiệu năng và độ ổn định của hệ thống. Cuối cùng, dựa trên kết quả thu được, hãy tối ưu API backend, rồi quay lại bước đầu để thiết kế lại các bài kiểm thử và lặp lại quá trình này cho đến khi bạn hài lòng với kết quả.

Mô tả cách bạn sẽ triển khai một chiến lược loại bỏ cache (cache eviction) ở phía máy chủ.

Để xác định chiến lược này, bạn cần làm rõ một số yếu tố sau: giới hạn kích thước sẽ kích hoạt việc loại bỏ dữ liệu khỏi cache khi bị vượt quá; một chiến lược giám sát để đánh giá xem cơ chế loại bỏ cache có hoạt động hiệu quả hay cần điều chỉnh; và một cơ chế vô hiệu hóa cache. Bên cạnh đó, bạn cũng cần lựa chọn chính sách loại bỏ (eviction policy) phù hợp, chẳng hạn như LRU (Least Recently Used) – loại bỏ các mục ít được truy cập gần đây nhất, LFU (Least Frequently Used) – loại bỏ các mục có tần suất truy cập thấp nhất, FIFO (First-In, First-Out) – loại bỏ các mục theo thứ tự được thêm vào, Random – loại bỏ ngẫu nhiên các mục, hoặc TTL (Time-To-Live) – tự động hết hạn các mục sau một khoảng thời gian nhất định.

Correlation ID là gì, và chúng có thể được sử dụng như thế nào để theo dõi (tracing) các yêu cầu xuyên suốt nhiều dịch vụ?

Correlation ID là các định danh duy nhất được gắn vào mỗi request trong các kiến trúc phân tán nhằm giúp việc theo dõi request xuyên suốt toàn bộ hệ thống. Thông thường, khi một request đi vào một hệ thống backend phân tán, dữ liệu của request đó sẽ phải đi qua nhiều dịch vụ web khác nhau trước khi tạo ra phản hồi cuối cùng. Nhờ có Correlation ID, việc hiểu rõ “hành trình” mà mỗi request đã trải qua trở nên dễ dàng hơn, từ đó hỗ trợ hiệu quả cho việc debug các vấn đề tiềm ẩn hoặc phân tích và tối ưu hiệu năng hệ thống.

Giải thích sự khác nhau giữa khóa lạc quan (optimistic locking) và khóa bi quan (pessimistic locking), và khi nào nên sử dụng từng loại.

Optimistic locking là một chiến lược kiểm soát đồng thời giả định rằng xung đột dữ liệu xảy ra khá hiếm. Chiến lược này cho phép nhiều tiến trình truy cập dữ liệu đồng thời và chỉ kiểm tra xem có xung đột hay không ngay trước khi thực hiện commit. Vì vậy, optimistic locking đặc biệt phù hợp với các hệ thống có tần suất đọc cao nhưng ghi thấp. Ngược lại, pessimistic locking là chiến lược giả định rằng xung đột dữ liệu xảy ra thường xuyên, nên dữ liệu sẽ bị khóa ngay từ đầu để ngăn chặn truy cập đồng thời. Các khóa này được giữ trong suốt vòng đời của transaction, do đó pessimistic locking phù hợp hơn với các kịch bản có tần suất ghi cao hoặc khi tính toàn vẹn dữ liệu là yếu tố quan trọng hàng đầu.

Bạn sẽ sử dụng những phương pháp nào để ngăn chặn deadlock (khóa chết) trong các giao dịch cơ sở dữ liệu?

Có nhiều cách để ngăn chặn deadlock trong các transaction của cơ sở dữ liệu, trong đó phổ biến nhất là áp dụng thứ tự khóa (lock ordering) để các khóa luôn được cấp phát theo một trật tự toàn cục nhất quán, từ đó tránh tình trạng chờ vòng tròn. Bên cạnh đó, việc sử dụng timeout cho các transaction cơ sở dữ liệu giúp tự động hủy những thao tác chạy quá lâu, giảm nguy cơ dẫn đến deadlock. Ngoài ra, khi có thể, nên ưu tiên sử dụng

cơ chế kiểm soát đồng thời lạc quan (optimistic concurrency control) để tránh việc giữ khóa trong thời gian dài.

Bạn sẽ bảo mật giao tiếp giữa các dịch vụ (inter-service communication) như thế nào trong một kiến trúc microservices?

Bắt đầu từ nguyên tắc rằng việc giao tiếp giữa các dịch vụ chỉ nên diễn ra trong các mạng riêng (lý tưởng nhất là không có lưu lượng truy cập công khai nào có thể tiếp cận các dịch vụ này), có một số khuyến nghị quan trọng cần lưu ý. Trước hết, hãy sử dụng các kênh truyền được mã hóa như TLS để ngăn chặn những hình thức tấn công phổ biến, chẳng hạn như tấn công trung gian (man-in-the-middle). Bên cạnh đó, nên dùng một API Gateway để quản lý và xác thực lưu lượng truy cập đi vào mạng riêng này. Cuối cùng, cần áp dụng chặt chẽ các cơ chế xác thực và phân quyền cho các thông điệp giữa các dịch vụ, đảm bảo rằng chỉ những microservice hợp lệ mới có thể giao tiếp với nhau, và mỗi dịch vụ chỉ được truy cập đúng phạm vi tài nguyên cần thiết.

Thảo luận về các kỹ thuật để ngăn chặn và phát hiện các bất thường dữ liệu (data anomalies) trong các hệ thống quy mô lớn.

Một số kỹ thuật phổ biến bao gồm việc bổ sung và triển khai các quy tắc kiểm tra dữ liệu nhằm ngăn chặn việc nhập dữ liệu không hợp lệ. Thông qua việc định nghĩa schema và các ràng buộc trên schema để áp đặt những tiêu chuẩn tối thiểu, bạn có thể phòng tránh các bất thường về dữ liệu xảy ra. Ngoài ra, việc triển khai cơ chế versioning cho dữ liệu giúp dễ dàng khôi phục lại trạng thái trước đó nếu phát hiện ra bất kỳ bất thường nào. Cuối cùng, cần xây dựng và duy trì các quy trình đảm bảo chất lượng dữ liệu chặt chẽ, nhằm đảm bảo mọi thông tin đi vào hệ thống đều được kiểm tra, xác thực đầy đủ và được đánh dấu khi cần thiết.

Mô tả quy trình tạo ra một giải pháp lưu trữ dữ liệu toàn cầu, có tính sẵn sàng cao (high availability) cho một ứng dụng đa quốc gia.

Việc xây dựng một hệ thống lưu trữ dữ liệu có tính sẵn sàng cao (highly available) đòi hỏi phải xem xét nhiều khía cạnh khác nhau. Trước hết là môi trường đa vùng (multi-zone): nếu bạn sử dụng các giải pháp đám mây như Azure, AWS hay GCP, yêu cầu này thường đã được đáp ứng sẵn (ngoại trừ một số khu vực đặc thù), nhằm đảm bảo hệ thống vẫn hoạt động ngay cả khi xảy ra sự cố mạng cục bộ. Bên cạnh đó, cần triển khai cơ chế nhân bản dữ liệu giữa các máy chủ ở tất cả các vùng để bảo đảm không bị mất dữ liệu khi một số máy chủ hoặc thậm chí cả một vùng gặp sự cố. Việc cân bằng tải cũng rất quan trọng, giúp phân phối lưu lượng truy cập hợp lý giữa các vùng sẵn sàng, từ đó giảm độ trễ cho người dùng. Ngoài ra, còn có những yêu cầu khác như thiết lập chính sách quản trị dữ liệu phù hợp để kiểm soát quyền truy cập, đồng thời đảm bảo tuân thủ đầy đủ các quy định pháp lý về dữ liệu tại địa phương, chẳng hạn như GDPR.